Variables, Control Structures and Calculations

- Every programming language, like any other language, has rules which must be followed.
- The rules that define the legal sequences of symbolic elements in a language is referred to as **syntax**.
- The syntax rules define the form of the various constructs in the language.
- A language construct is defined as a syntactic structure or set of structures in a language to express a particular class of operations.
- **Reserved words** and **keywords** are the building blocks in a programming language which have a special meaning in the programming language.
  - Examples of keywords are: PRINT, INPUT etc.
  - Examples of reserved words are: IF, REPEAT etc.

1

Pseudo code

- The work in this course is concerned with concepts associated with programming rather than a particular programming language.
  - Therefore, pseudo code will be used.
- Pseudo code is one of the tools that is used to write a preliminary plan which is developed into a computer program.
- It is not a standard language, although programmers often use terms within it that closely resemble the actual language to be used.
- Its purpose is to describe an algorithm in a form that can easily be understood and then translated into the actual programming code required.
- The syntax of the language to be used and the fine detail of the program are ignored until writing the source code, i.e. the program, which will be compiled later.

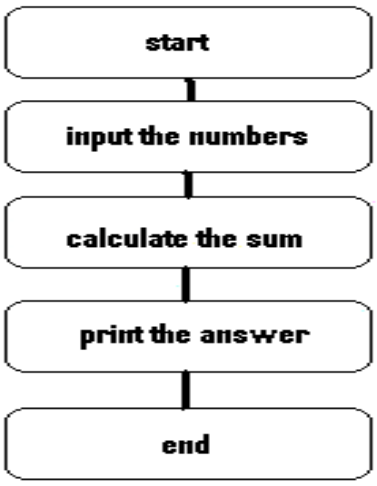
2

Cont..

- Diagrams may also be used to represent an algorithm. The diagrams are similar to flowcharting, although the official standards are not used.
- First problem considered: -
- Write the following algorithm using a diagram: input two numbers from the keyboard of a personal computer, add them together and output the result.
  - This can be thought of as a more general problem in that data is accepted, a process is performed on the data and the results are returned to the user.

3

Solution in form of a diagram: -



4

Cont..

- There must be a method of accepting data and storing it in the computer's memory.
- The computer program also needs to know where the data is stored in memory so that it can process the data.
- Programming languages use variables for this purpose.

5

Variable Types and Names

- Variables are used to store data in the computer's memory (RAM) during the time that the program is running.
- Three attributes are associated with variables: -
  - The address in memory where the data is stored;
  - The actual data stored which can change during the execution of the program;
  - The name of the variable or identifier.
- In the example to calculate the sum of two numbers, three variables are needed.
- It is good practice to think of names which are meaningful in the context of the program
  - This helps in understanding the coding.

6

Cont..

- In this case, number1, number2, and sum could be used.
- The pseudo code could be written as: -
-
- ***ACCEPT*** *number1, number2*
- *sum:= number1 + number2*
- ***PRINT*** *sum*
- *end program*
- The words ACCEPT and PRINT can be thought of as reserve words.
- When a variable is declared, an address in memory is allocated to it and the value input through the keyboard is stored in this memory location.

7

Cont..

- For example, when number1 is declared as a variable, an address in memory is allocated to it and the number input through the keyboard is stored in this memory location.
- Every time the identifier number1 is used in the program, this memory location would be accessed and thus the data stored in it can be read and manipulated.

8

Choice of variables names

- There are certain limitations on the names you can use for variables.
- The name chosen must conform to certain rules of the programming language used:
  - Words that make up the programming language cannot be used as variable names. These words are the reserved words which have a special meaning in the programming language.
- Punctuation marks and spaces may also have a special meaning within the grammar or syntax of a programming language you are learning, so these cannot be used as part of a variable name, either.

9

Variables types

- **Compiler**
  - A program that translates high-level language into absolute code, or sometimes into assembly language.
  - The input to the compiler (the source code) is a description of an algorithm or program in a problem-oriented language; its output (the object code) is an equivalent description of the algorithm in a machine-oriented language.

10

Cont..

- **Interpreter**
- A language processor that analyses a line of code and then carries out the specified action, rather than producing a machine-code translation to be executed later.
- Computer programs need to be compiled or interpreted before they are run to get the required output from the programs.
- The compiler or interpreter needs to know what type of data is being used in a program.
  - Compilers/interpreters have a different way of dealing with whole numbers, fractional numbers and characters.
  - The type of data being used has a great influence on program design.

11

Cont..

- Different types of data used in programming languages are represented as follows:
  - Whole numbers (e.g. 1, 678) – Integer
  - Fractional numbers (e.g. 3.678, 789.3) – Real or Float
  - Letters (e.g. a, f, z, 4) – Character
  - Names (e.g. Dr. H. Kelly) – String
  - FALSE or TRUE – Boolean (only 2 values allowed)

12

Variable declaration

- Many programming languages insist on a variables definition at the beginning of the program.
- This emphasizes the importance of data in deciding the structure of the program.
  - Therefore, it is good practice to identify the variables used and the types in the pseudo code.
  - It is also good practice to describe any printed or displayed output, rather than just printing a number.
  - Note that the output and input to the program are the human-computer interface and as such must be meaningful and helpful to the user.

13

Cont..

- The pseudo code for the previous problem now becomes:

```
program start
Use variables: number1, number2,
sum OF TYPE Integer
DISPLAY "Enter two whole numbers"
        ACCEPT number1, number2
        sum:= number1 + number2
        PRINT "The sum is" sum
    end program
```

14

Constants

- Recall that that variables are used to hold data which can change throughout the program.
- But some data stays the same throughout the running of the program.
- Data items which have the same value throughout the program are termed as constants.
- As with variables, the computer needs a name for each constant and to know its data type so that it can be manipulated as required during the program.

15

Operators for arithmetic

- Computer languages provide a range of operators for arithmetic. Common operator used in programming languages are:
  - + meaning add
  - – meaning subtract
  - * meaning multiply
  - / meaning divide

16

Order of precedence

- The order in which a calculation is evaluated is very important since the end result can differ according to which operation is given precedence, i.e. performed before another.
- Having rules for the order of precedence of the arithmetic operations avoids ambiguity when arithmetic expressions are evaluated.
- The way expressions are worked out by a computer are very similar to the rules in mathematics, and are as follows:

do *operations in brackets first;*
then *evaluate any exponentiation (e.g. finding the cube of a number, work out 6 to the power of 4);*
then *any multiplication or division operation;*
and *lastly addition and subtraction.*

17

Cont..

- **Example:** an employee pays 20% of his/her salary as income tax to the government. Write a pseudo code to calculate the amount paid to the government as his/her income tax.

```
program start
Uses variable: salary, AmountPaid  OF TYPE real
                IncomeTax := 20
DISPLAY "Enter the salary for the employee"
ACCEPT salary
        AmountPaid := (IncomeTax * salary)/100
        PRINT "The income tax paid is" AmountPaid
End program
```

18

Control Structures

- A syntactic form in a language to express flow of control of execution in a program.
- There are three main types of instructions, i.e. control structures, used in programming: -
  - Sequence
  - Selection
  - Loops/repetitions.

19

Sequence

- Involves executing instruction in a sequence, i.e. step by step.
- The problems met so far have all included instructions which have to be carried out in sequence.

20

Selection

- Use where a program is required to include a range of alternative actions.
- Programming languages provide selection using: -
  - IF…THEN statement
  - CASE statement

21

IF…THEN statement

- IF…THEN statement is used together with logical operators (such as equal, less than) to test for true or false condition of statement.
- The logical operators used in pseudo code are:
  - =: is equal to
  - >=: is greater than or equal to
  - <=: is less than or equal to
  - <>: is not equal to
  - > : greater than

22

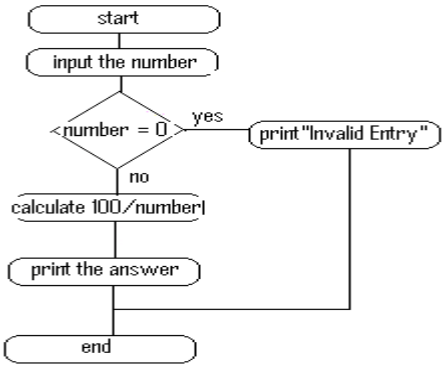
Cont..

- Example, write a pseudo code for a program in which a 100 (a constant) is divided by number (a variable).
- Note that it is possible that the variable could contain a zero, particularly if the data was entered via the keyboard and the operator had made a mistake.
- As it is not possible to divide by zero, the calculation should not be performed on this data, so the condition would have to be checked first.

23

Cont..

- The logic of the sequence of instructions needed can be illustrated in the form of a flowchart:



24

Cont..

- Its pseudo code is as follows:

```
program start
        uses variables : number OF TYPE integer
                         quotient OF TYPE real
                number1 = 100
        DISPLAY "Enter any integer number"
         ACCEPT number
         IF number <> 0 THEN
                Quotient := number1 / number
    DISPLAY number1 "divided by" number
        "is" quotient
        ENDIF
         IF number = 0 THEN
                DISPLAY "Invalid Entry"
         ENDIF
end program
```

25

Cont..

- Note that the instructions following the THEN part of the IF statement are executed if the condition is TRUE.

26

Cont..

- **Another Example:**
- Write a pseudo code for a problem to offer a user a menu of choices and allows the input of two numbers and the calculation of the sum, difference or product of the numbers, based on the choice made by the user.
- An example of screen display when the program is run is shown below:

```
Choose one of the following:
    m for multiply
    a for add
    s for subtract
Input the numbers you want to use
    21 63
```

27

Extending the IF…THEN statement

- The IF…THEN statement can be extended by adding an ELSE statement to the IF…THEN statement.
- The IF…THEN…ELSE allows a selection between two alternatives, depending on the truth or falsity of a given condition.
- Example, write the example of checking division by zero using the IF…THEN…ELSE statement.

28

The CASE statement

- Repeating the IF ... THEN ... ELSE statements a number of times can be somewhat confusing.
- A cleaner (and easier to follow) construct is to use a selection method that specifically covers the set of alternative conditions that are needed.
- This is achieved by a CASE statement.
- The CASE statement is frequently used for coding the choice between items in lists.

29

Cont..

- Example, Write the pseudo code for a program to input a category of insurance and display details of the type of insurance available. Use the CASE statement.

Category	Type of Insurance
U	Insurance not available
A	Insurance is doubled
B	Insurance is normal
M	Insurance is medically independent

- Any other category entered is invalid.

30

Cont..

- Solution:

```
Program start
        Use variables:  category OF TYPE Character
          DISPLAY "Enter category of insurance"
        ACCEPT category
        DO CASE of category
           CASE category = "U"
             PRINT "Insurance not available"
           CASE category = "A"
             PRINT "Insurance is double"
           CASE category = "B"
             PRINT "Insurance is  normal"
           CASE category = "M"
             PRINT "Insurance is medically dependent"
           OTHERWISE PRINT "entry is invalid"
        ENDCASE
End of program
```

31

Cont..

- Note that the last alternative statement in the CASE statement is OTHERWISE.
- This statement will be executed if no other CASE has been selected as TRUE.
- It is always advisable to include the OTHERWISE case in every CASE statement.

32

Logic operations

- There are many occasions when the number of conditions tested is extended in a single IF…THEN statement.
- Often these conditions are linked in a logical way, and as such are called logical operations.
- Conditions in IF…THEN statements are linked using an **AND** and an **OR**.
- Conditions linked with an AND only result in an action when all conditions are true.
- For example: IF a > b AND a > c THEN display “a is the largest”.
- Conditions linked with an OR lead to an action when either condition is true.

33

Cont..

- Example:
- Write a pseudo code for a problem to input an examination mark and test it for the award of a grade.
- The mark is a whole number between 1 and 100. Grades are awarded according to the following criteria:
  - >= 80 distinction
  - >= 60 and < 80 merit
  - >= 40 and < 60 pass
  - < 40 fail

34

Cont..

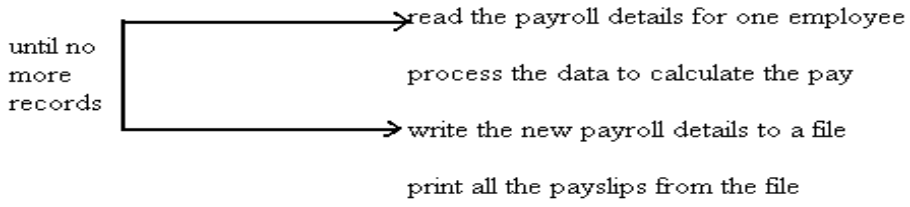
- Pseudocode solution:

```
Program start
        Use variables: mark OF TYPE Integer
        ACCEPT mark
        IF mark >= 80 THEN
display "distinction"
        ENDIF
        IF mark >= 60 AND mark < 80 THEN
display "merit"
        ENDIF
        IF mark >= 40 AND mark < 60 THEN
display "pass"
        ENDIF
        IF mark < 40 THEN
display "fail"
        ENDIF
    end program
```

35

Loops

- Used to control execution of a sequence of instructions which are repeated a number of times until a certain prescribed condition is satisfied.
- Loops are also referred to as iterations, but strictly speaking an iteration is more complex than a simple loop.
- The instructions needed to process all the payroll records of employee information on a file and then print pay slips are an example of a loop:



36

Cont..

- The three instructions above are in a loop and they will be repeated for every record on the payroll file.
- The loop will terminate when end-of-file is reached.
- When the loop terminates, the instruction after the loop is executed, in this example, 'print all the pay slips from the file' would be executed.
- An iteration is a repetition of a sequence of instructions where the results from one pass of the loop are used as input to the instructions in the next pass of the loop.

37

Cont..

- An example of an iteration would be if, during the running of the payroll loop above, a running total was kept of e.g. tax paid.
  - Each time through the loop, the tax for the current employee would be added to the old current total which would then be used as input in the following loop for the next employee.

38

Cont..

- A loop has two main elements: -
  - A sequence of instructions to be performed each time the loop is executed;
  - An indication of when to stop executing the loop.
    - This part contains a logical test as a certain condition will need to be checked.
    - This checking can be done at different stages of the loop, e.g. at the beginning or at the end, depending on the type of instruction used to control the loop.

39

Cont..

- There are three constructs for loops in our pseudo code:
  - REPEAT…UNTIL, which tests the condition for exiting a loop at the end of a block of code.
    - Consequently, sequence of instructions is always executed at least once.
  - WHILE, which tests the condition for exiting a loop at the start of a block of code.
    - Consequently, it is possible that the instructions in the loop may never be executed.
  - FOR loop, which is controlled by a count given from known conditions?

40

REPEAT…UNTIL

- It has the following structure: -

  REPEAT

  <statement or a block of statements>

  UNTIL <a true condition occurs>

- For example, percentage examination marks could be terminated by a number outside the range of 0 to 100.

41

Cont..

- Consider a program segment to allow entry of a number in the range 0 to 100.
- Its pseudo-code solution is as follows:

```
program start
    Use variables: number of type Integer
    REPEAT
      DISPLAY "Enter a number between 0 and 100"
      ACCEPT number
    UNTIL number <0 or number > 100
end program
```

- The sequence of instruction in the pseudo code can also be written in form of a flow chart.

42

The WHILE LOOP

- It has the following structure:

```
WHILE <a true condition> DO
    <statement or block of statements>
ENDWHILE
```

- For example, a program segment to display each character typed at a keyboard until the character q is entered implemented using the WHILE loop is as follows: -

```
program start
   Use variables: letter OF TYPE Character
        DISPLAY "Enter any character"
      ACCEPT letter
WHILE letter <> "q" DO
          DISPLAY "the character you typed is ", letter
             DISPLAY "Enter any character"
          ACCEPT letter
      ENDWHILE
  end program
```

43

FOR LOOP

- Used when the number of repetitions is known in advance.
- It uses two values of a variable, the starting state and the final state for controlling the loop.
- The variable is either incremented or decremented during each iteration until it reaches the value identified as the final state.
- It has the following structure: -

```
FOR (starting state, final state, increment) DO
<statement or a block of statements>
ENDFOR
```

44

Cont..

- The loop is controlled by a variable which is a simple count and is assigned a starting value.
- Each pass through the loop involves the following instructions:
  - compare current value of count to final state.
    - If count has passed the final state (i.e. count > final state) then terminate, otherwise continue;
  - add the increment to the count.
- The general pseudo code definition for a FOR loop can be:

  FOR (n:= a, n = b, +1) DO

  <command sequence>

  ENDFOR

  - where **a** is the starting point, **+1** is the increment and **b** is the final value.
  - This means that the loop will continue to be executed while **n** is less than or equal to **b**.

45

Cont..

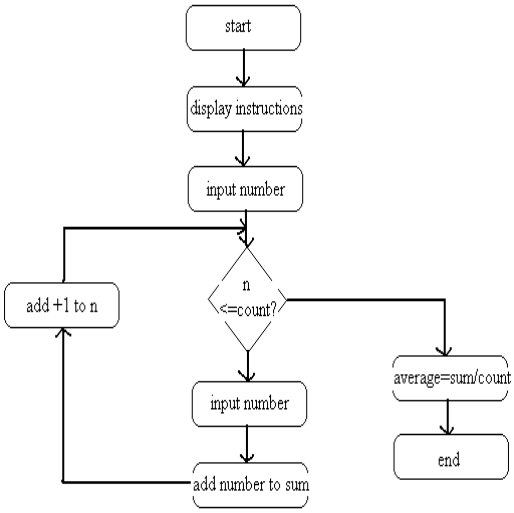
- For Example, write a program to calculate the sum and average of a series of numbers.
- The user will decide how many numbers are entered.
- The algorithm (solution) for this program is shown below as pseudo code:

46

```
 program start
Use variables: n, count OF TYPE Integer
                        sum, average, number OF TYPE Real
DISPLAY "How many numbers do you want to
          input?"
          ACCEPT count
            sum:= 0;
          FOR (n:=1, n = count, +1)
               DISPLAY "Enter number"
             ACCEPT number
             sum := sum + number
          ENDFOR
          average:= sum/count
          DISPLAY "The sum of the numbers was ", sum
DISPLAY "The average of the numbers was",
           average
       end of program
```

47

Flowchart



48